

UNIVERSIDADE FEDERAL DE SANTA CATARINA
CENTRO TECNOLÓGICO
DEPARTAMENTO DE INFORMÁTICA E ESTATÍSTICA

**ARTHUR ERPEN
CAETANO PERUZZO
GABRIEL HERRERA**

Relatório sobre a Etapa 5 de Sistemas Operacionais II

Florianópolis
2026

Resumo

Este relatório descreve a implementação da Etapa 5 do projeto de Sistemas Operacionais II: a comunicação *Time-Triggered Publish-Subscribe* por mensagens de **Interesse** e **Resposta**, sem publicação explícita. Sobre a pilha das etapas anteriores (Communicator / Protocol / NIC / SPTP / filtro espacial) adicionamos uma camada nova, `SmartData<Tipo>`, parametrizada pelo tipo do dado. Um agente que quer um dado emite uma mensagem de Interesse, em *broadcast*, identificando o tipo por um código `Unit`, usado no papel dos identificadores TEDS (IEEE 1451); os agentes capazes de produzir aquele tipo armam uma `Periodic_Thread` e passam a enviar Respostas periódicas enquanto houver interesse. O ciclo de vida do interesse é gerido tanto pelo produtor (`Binding_Cache` com estado *soft-state* por endereço) quanto pela RSU, que atua como **broker** do quadrante: faz rastreamento passivo, reanuncia interesses e emite desinteresse — inclusive detectando a saída de um agente pela *presença* do SPTP, sem troca explícita de mensagem. A mobilidade é *implícita*: o cliente não conhece o quadrante; o filtro espacial da NIC corta os dados ao trocar de quadrante e o consumidor reanuncia reativamente. O texto apresenta os requisitos, a arquitetura, os detalhes de implementação e a validação experimental por uma suíte de 10 cenários executados em QEMU, culminando numa frota móvel de 24 VMs (4 RSUs, uma por quadrante, e 20 veículos livres).

Sumário

1	Introdução	2
2	Formato das mensagens	3
3	A classe <code>SmartData</code>	3
4	Lado produtor: <code>Binding_Cache</code> e threads periódicas	4
5	Lado consumidor: <i>refresh</i> reativo e modo-valor	5
6	A RSU como <i>broker</i> do quadrante	5
7	Mobilidade implícita	6
8	Desinteresse	7
9	Validação experimental	7
10	Resultados	8
11	Conclusão	8

1 Introdução

Nas etapas anteriores construiu-se uma API de comunicação capaz de transportar mensagens dentro de uma VM (memória compartilhada System V) e entre VMs (AF_PACKET), com *timestamp* sincronizado (SPTP) e um filtro espacial por quadrantes na NIC. A Etapa 5 acrescenta um *modelo de comunicação* sobre essa infraestrutura: a troca *publish-subscribe* disparada por tempo, porém **sem publicação explícita**.

No modelo, quando um agente quer um dado ele envia uma mensagem de **Interesse** (todas as mensagens são *broadcast*) que designa inequivocamente o *tipo* do dado. Os agentes que recebem o Interesse e são capazes de produzir aquele tipo passam a enviar, periodicamente, mensagens de **Resposta**. As estruturas seguem a especificação:

$$I = \{origin, timestamp, type, period\} \quad R = \{origin, timestamp, type, value\}$$

Requisitos da etapa:

- Comunicação por Interesse/Resposta, sem *publish* explícito.
- Todas as mensagens em *broadcast*.
- Cada mensagem carrega um código que identifica sua natureza e o tipo do dado — usamos o enumerado **Unit**, inspirado no papel dos códigos TEDS da norma IEEE 1451, para essa identificação inequívoca.
- O Interesse especifica o *período* das respostas, e cada produtor pertinente gerencia o intervalo de forma correspondente.
- Comunicação orientada a eventos não está prevista (apenas *time-triggered*); a sincronização temporal é presumida (SPTP de fundo).

A camada nova encaixa-se acima do **Communicator**, como mostra a Figura 1. Não foi necessário alterar a rede: o **SmartData** e o *broker* da RSU usam o **Communicator** já existente.

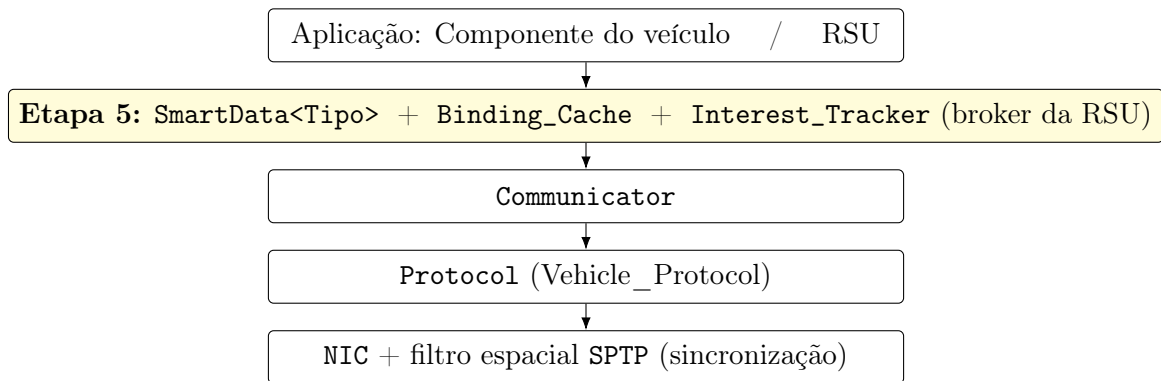


Figura 1: Posição da camada da Etapa 5 na pilha. Apenas a faixa destacada é nova; o restante vem das etapas anteriores.

2 Formato das mensagens

As estruturas que viajam no *payload* de uma `Message` estão em `smart_message.h`. O cabeçalho `SmartHeader` carrega dois campos: `kind`, que identifica a *natureza* (Interesse ou Resposta), e `unit`, o identificador inequívoco do tipo do dado. O Interesse acrescenta o período e um bit de desinteresse; a Resposta acrescenta o valor, parametrizado pelo tipo.

```
struct SmartHeader {
    enum Kind : uint8_t { INTEREST = 0, RESPONSE = 1 };
    uint8_t kind;
    Unit    unit;                // identificador do tipo do dado
} __attribute__((packed));

struct InterestMessage {
    SmartHeader header;
    uint64_t    period_us;       // período pedido para as respostas
    uint8_t     disinterest;     // 1 = cancela o interesse
} __attribute__((packed));

template <typename Value>
struct ResponseMessage {
    SmartHeader header;
    Value        value;
} __attribute__((packed));
```

Listing 1: Estruturas de mensagem (`smart_message.h`).

Os campos *origin* e *timestamp* da especificação **não** são replicados no *payload*: eles já viajam no cabeçalho do `Protocol`, preenchidos pelo `Communicator` no momento do envio (o *timestamp* pelo `Clock::monotonic_stamp` da Etapa 3). Todas as estruturas são `packed`, pois cruzam a fronteira de VMs. O conjunto de tipos (`Unit`) é um enumerado: cada valor é um identificador estável, inspirado no papel dos códigos TEDS, associado a um descritor que define o tipo do `Value` (e.g. `Speed_Data`, `Counter_Data`), de modo que o `SmartData` possa ser parametrizado por tipo.

3 A classe `SmartData`

O núcleo da etapa é `SmartData<Tipo>`, uma classe parametrizada pelo tipo do dado: cada tipo (velocidade, distância, contador...) instancia a classe com seu próprio `Value`. A mesma classe assume **dois papéis**, escolhidos pelo construtor:

- **Produtor**: recebe um `IProducer<Value>*` — a interface pela qual o componente fornece o valor. Ao receber um Interesse, registra o *binding* e arma uma thread periódica.
- **Consumidor**: recebe o *período*. No construtor já emite o Interesse; depois lê as Respostas.

A recepção é **push**: `SmartData` herda `Concurrent_Observer<Message>` e observa o `Communicator`. O `Communicator`, por sua vez, é um `Concurrent_Observer<Message>` (notifica) e um `Channel::Observer` (observa o `Protocol`). Quando um quadro chega, o

Communicator faz o *unmarshal* para uma *Message* e notifica os *SmartData* interessados; o método *update()* filtra por *kind* e *unit*:

```
void update(Port c, Message * m) override {
    const SmartHeader * h = header_of(m);
    if (!h || h->unit != UNIT) { delete m; return; }    // demux por Unit

    if (_role == PRODUCER && h->kind == SmartHeader::INTEREST) {
        const InterestMessage * im = /* ... */;
        if (im->disinterest) _cache->on_disinterest(UNIT, m->address());
        else
            _cache->on_interest(UNIT, m->address(), im->
period_us);
        delete m;
    } else if (_role == CONSUMER && h->kind == SmartHeader::RESPONSE) {
        _last_response_ns.store(Clock::now_ns());    // alimenta o refresh
reativo
        /* enfileira a Message (modo fila) OU cacheia o valor (modo-valor) */
    } else {
        delete m;
    }
}
```

Listing 2: Despacho no *update()* do *SmartData* (resumido).

Reaproveitamos, assim, o padrão *Observer/Observed* das etapas anteriores: o canal *empurra* a mensagem até o *SmartData*, e a aplicação apenas lê o resultado.

4 Lado produtor: *Binding_Cache* e threads periódicas

Quando um produtor recebe um Interesse, ele registra o pedido no *Binding_Cache*. Cada *binding* guarda o **endereço** de quem demonstrou interesse, de modo que um desinteresse de um agente remova apenas o registro *dele*, sem derrubar os demais interessados na mesma *Unit*. Como as respostas são *broadcast*, mantém-se **uma única** *Periodic_Thread* por *Unit*, respondendo no período **mais curto** entre os interessados:

```
void on_interest(Unit unit, const Address & addr, uint64_t period_us) {
    std::lock_guard<std::mutex> lock(_mtx);
    Binding * b = find_binding(unit, addr);
    if (b) { b->period_us = period_us; b->expiry_ns = expiry_for(period_us); }
    else    _bindings.push_back({unit, addr, period_us, expiry_for(period_us)});
    resync_threads();    // 1 thread por Unit, no periodo mais curto
}

void on_disinterest(Unit unit, const Address & addr) {
    std::lock_guard<std::mutex> lock(_mtx);
    // remove apenas os registros DESTES endereço; os outros seguem atendidos
    _bindings.erase(remove_if(/* unit && addr */, _bindings.end()));
    resync_threads();
}
```

Listing 3: Registro de interesse e ressincronização das threads (*binding_cache.h*).

O estado é *soft-state*: cada *binding* tem validade (*expiry*), renovada quando o Interesse é reouvido; um *reaper* em segundo plano remove os *bindings* expirados. Assim, um interessado que simplesmente desaparece é esquecido mesmo sem enviar desinteresse. A thread periódica chama `respond()`, que pede o valor ao componente e o envia:

```
void respond(Unit) {
    ResponseMessage<Value> r;
    r.header.kind = SmartHeader::RESPONSE;
    r.header.unit = UNIT;
    r.value = _producer->produce();           // o COMPONENTE gera o dado
    TypedMessage<ResponseMessage<Value>> msg(r);
    _comm->send(&msg);                       // broadcast; o timestamp e
    carimbado aqui
}
```

Listing 4: Envio de uma Resposta (`respond`).

Adicionalmente, mesmo que um nó interessado permaneça em um quadrante sem enviar mensagens periódicas de interesse, ele continua recebendo os dados solicitados. Isso é garantido pelas funcionalidades de retransmissão de interesse e detecção de saída implementadas na RSU. Mais sobre isso na seção 6

5 Lado consumidor: *refresh* reativo e modo-valor

Como uma única mensagem de Interesse pode se perder (nem todos a recebem), o interesse precisa ser renovado. Em vez de *polling* fixo, adotamos um **refresh reativo**: o consumidor só reenvia o Interesse após um *silêncio* de dados (variável `_last_response_ns`). Em fluxo estável ele fica quieto; quando os dados cessam — por perda ou por troca de quadrante — ele reanuncia. Esse mecanismo é também a base da mobilidade implícita (Seção 7).

A leitura tem **dois modos**:

- *Modo fila* (padrão): cada Resposta é enfileirada como `Message` e a aplicação a drena com `receive_response()`.
- *Modo-valor* (*opt-in*): o consumidor não enfileira; guarda apenas o **último valor**, lido como uma variável viva por `value()` / `operator Value()`, com validade simples (`fresh()` / `expired()`, TTL = 3× período). É o modelo SmartData/EPOS, em que o dado distribuído é lido como se fosse local.

6 A RSU como *broker* do quadrante

A RSU, sendo o nó fixo do quadrante, hospeda um `Interest_Tracker` que faz **rastreamento passivo**: ouve os Interesses, **reanuncia** periodicamente (cobrindo perdas e veículos que entram tarde) e **manda parar** quando uma `Unit` fica sem interessados. O ponto-chave é que a saída de um veículo é detectada pela **presença** do SPTP, sem mensagem extra: o *master* dispara `on_presence(mac)` a cada `REQUEST_SYNC`, e o broker mantém um *lease*.

```
void broker_tick() {
    // (1) purga veiculos cujo lease de presenca expirou + seus interesses
```

```

for (mac : presencas_velhas) { _presence.erase(mac); remove_interesses(mac);
}
// (2) conjunto ativo: Unit -> periodo mais curto
// (3) Units que sumiram desde o tick anterior -> desinteresse
for (Unit u : sumiram)    send_msg(u, 0,          /*disinterest=*/true);
for (Unit u : ativas)     send_msg(u, periodo,    /*disinterest=*/false); //
reanuncia
}

```

Listing 5: Tick periódico do broker (`interest_tracker.h`, resumido).

O broker fica **sempre ativo** (um por quadrante). Quando há vários RSUs (um por quadrante), cada um é o *master* SPTP e o broker do seu quadrante; o filtro espacial isola o tráfego, de modo que os quatro *masters* coexistem sem se enxergar.

7 Mobilidade implícita

Uma decisão central de projeto foi **não dar GPS nem noção de quadrante ao componente**, evitando uma quebra de camada. A mobilidade emerge da combinação do filtro espacial da NIC com o refresh reativo do consumidor, como na Figura 2: ao trocar de quadrante, os quadros do produtor antigo são descartados pelo filtro; o consumidor entra em silêncio e reanuncia; a NIC carimba o quadrante *atual* no reenvio; e um produtor co-localizado no novo quadrante assume. Do lado do produtor, a entrada num novo quadrante dispara `Binding_Cache::clear()`, descartando os interesses do quadrante que ficou para trás.

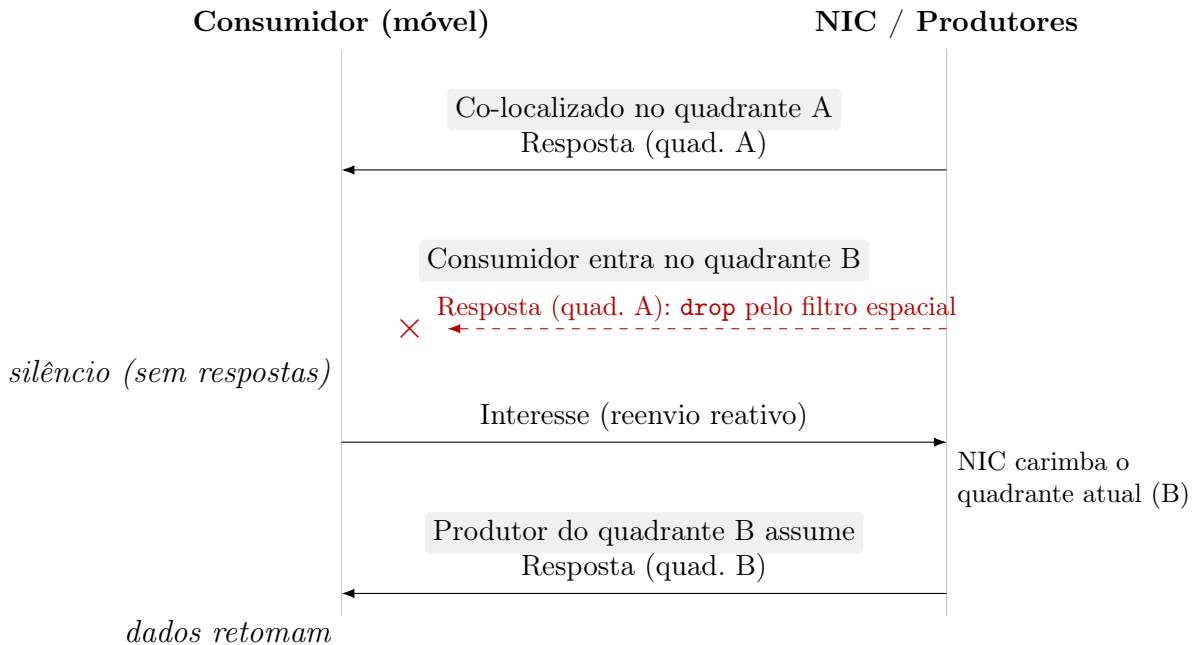


Figura 2: Handover implícito de quadrante: o filtro corta os dados, o consumidor reanuncia (re-carimbado), e o produtor do novo quadrante assume.

8 Desinteresse

O fim do interesse é tratado por dois caminhos complementares:

- **Explícito (bit):** quando o consumidor sai de forma educada, envia um Interesse com `disinterest=1`; o produtor remove **apenas o binding daquele endereço** (*per-address*).
- **Por presença (RSU):** quando o consumidor some sem avisar, o *lease* de presença expira na RSU e o broker emite o desinteresse aos produtores.

Em ambos os casos o produtor para de gastar banda. A reaper *soft-state* do produtor é a terceira rede de segurança, expirando *bindings* não renovados.

9 Validação experimental

A validação usa o mesmo arcabouço das etapas anteriores: cada binário descobre seu papel por `so2.vm_id` no `/proc/cmdline`; o harness (`run_qemu_test.sh`) sobe N VMs em QEMU e exige a linha "cenario validado." em *todas*. As asserções são por **contagem de produtores distintos / origem** (chave MAC+porta), não por *timing* frágil; `decode_response` valida o tipo (`kind+unit`). Utilitários comuns (`detect_vm_id`, `decode_response`, `endpoint_key`) ficam em `interest_test_utils.h`. Foram escritos **10 cenários**:

- **fanin** — 1 consumidor recebe de N produtores (caso fundamental).
- **fanout** — N consumidores em *dois* períodos, 1 produtor; prova a agregação numa única thread no período mais curto.
- **types** — quatro tipos simultâneos; cada consumidor só aceita a sua **Unit** (demux por tipo).
- **period** — aderência ao período com 20 produtores.
- **lifecycle** — desinteresse explícito + *per-address* (um consumidor sai, outro continua sendo atendido).
- **presence** — saída detectada pela presença do SPTP, sem bit.
- **rsu-repeat** — produtor que entra tarde é servido pelo reanúncio da RSU.
- **mobility** — frota móvel (9 VMs) com *roaming* e reanúncio por quadrante.
- **scale** — frota móvel de 24 VMs: **4 RSUs** (uma fixa por quadrante) e **20 veículos livres**, com quatro tipos circulando.
- **value** — leitura pelo `operator Value()` (modo-valor).

O cenário **scale** merece destaque por exercitar, de uma vez, quatro mecanismos: *broker* por quadrante (4 RSUs), filtro espacial, mobilidade implícita e demux por tipo. As VMs 1–4 são ancoradas em quadrantes distintos (`initial_quadrant`) e congelam a posição (a RSU, sendo *master*, fixa o quadrante via `ioctl GPS_IOC_SET_FIXED`); as demais 20 VMs andam livremente. Cada consumidor precisa agregar produtores do seu tipo ao reencontrá-los pelos quadrantes e reanunciar ao se separar.

10 Resultados

Todos os 10 cenários passam em QEMU. A Tabela 1 resume a topologia e a evidência principal de cada um.

Tabela 1: Resumo da suíte de testes da Etapa 5 (todos aprovados).

Teste	VMs	Evidência observada
fanin	7	consumidor recebe dos 5 produtores distintos
fanout	6	produtor mede ≈ 48 respostas/janela = 1 thread a 250 ms (não a soma 250+500); 4 consumidores atendidos
types	12	consumidores de unit = 2, 3, 4, 61441, cada um só com o seu tipo
period	22	20 produtores; pior média $\approx 500,994 \mu s$ para período pedido de 500 ms
lifecycle	7	após a saída de A, B vai de 168 para 273 respostas (per-address)
presence	3	produtor recebe desinteresse emitido pela RSU (presença)
rsu-repeat	3	produtor tardio servido; respostas = 3 via reanúncio
mobility	9	consumidores móveis: distintos = 2, reissues = 2-3
scale	24	4 RSUs + 20 móveis; consumidores com distintos = 2-6, reissues = 2-4, quatro tipos demuxados
value	3	valor avança de 4 para 24, sempre fresh

No **scale**, as 24 VMs validaram e cada um dos 20 consumidores móveis recebeu apenas o seu tipo, agregando de 2 a 6 produtores distintos ao percorrer os quadrantes — evidência de que o *broker* por quadrante, o filtro espacial e a mobilidade funcionam em conjunto e em escala.

11 Conclusão

A Etapa 5 entregou a comunicação *Time-Triggered Publish-Subscribe* pedida e foi além em três frentes. Cumprimos a especificação: Interesse/Resposta sem *publish*; tudo em *broadcast*; identificação por **Unit** inspirada nos TEDS; período gerenciado pelo produtor (com agregação na thread mais curta); e leitura do dado como variável (**operator Value()**). Acrescentamos o **rastreamento passivo na RSU** com desinteresse por *presença* do SPTP, a **mobilidade implícita** sem GPS no cliente, e **bindings per-address** que isolam o desinteresse de cada agente.

As principais decisões de projeto foram: o *push* fim-a-fim via Observer (o **Communicator** notifica o **SmartData**), aderente à ideia de “o protocolo dá um *notify*”; o estado *soft-state* que tolera saídas abruptas; e o *broker* sempre ativo, um por quadrante. A validação, de **fanin** (7 VMs) à frota móvel **scale** (24 VMs, 4 RSUs), confirma que o modelo se mantém correto desde o caso mínimo até uma frota realista em movimento.